

Teoria Współbieżności

Laboratorium 7

Wzorce projektowe dla programowania współbieżnego

Kacper Bieniasz

28 listopada 2024

1 Dane techniczne sprzętu

Obliczenia zostały wykonane na komputerze o następującej specyfikacji:

- Procesor: AMD Ryzen 7 5800U
- Pamięć RAM: 16 GB DDR4 3200 MHz (2×8GB)
- System operacyjny: Windows 11 Home x64

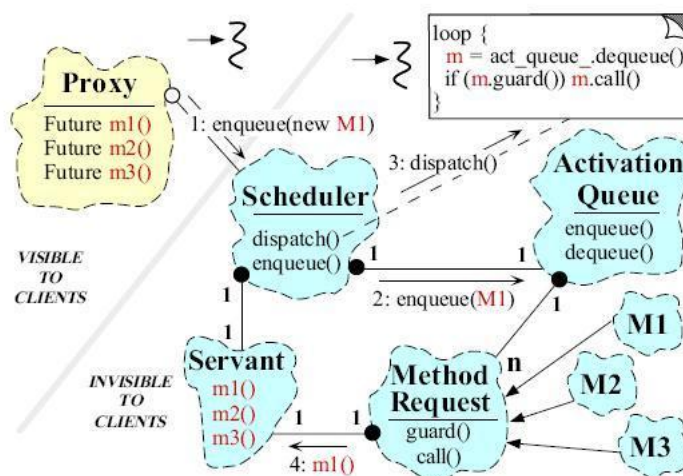
2 Treść polecenia

Zadanie polegało na wykorzystanie wzorca projektowego Active Object, w programie współbieżnym. Wzorec aktywny obiekt zaimplementować jako bufor w problemie producentów i konsumentów. Active Object składa się z 6 składników:

- **Proxy** (Pośrednik). Reprezentuje interfejs obiektu. Udostępnia interfejs dla klientów, przy pomocy którego wywołują oni publicznie dostępne metody aktywnego obiektu. Wywołanie takiej metody powoduje utworzenie obiektu Method Request i umieszczenie go w kolejce Activation Queue należącej do Schedulera.
- **Servant** (Pracownik). Dostarcza implementacji obiektu. Implementuje metody zdefiniowane w Proxy i odpowiednim Method Request. Metoda Servanta jest wywołana gdy odpowiadające jej żądanie metody (Method Request) jest wykonane przez Scheduler. Servant wykonuje się w wątku Schedulera. Servant może dostarczać dodatkowych metod, które mogą posłużyć do implementacji strażników w Method Request
- **Method Request** (Żądanie metody). Używane jest do przekazanie kontekstu wywołania metody (np. parametrów) z Proxy do Schedulera uruchomionego w osobnym wątku. Abstrakcyjna klasa Method Request definiuje interfejs dla wykonywania metod Aktywnego Obiektu. Interfejs ten zawiera również metody strażników (guard), które są mogą być użyte do sprawdzenia, czy warunki związane z synchronizacją są spełnione.
- **Activation Queue** (Kolejka aktywacji). Zawiera bufor oczekujących Method Request utworzonych przez Proxy. Jest zasobem dzielonym dla wątków klienta i pracownika - pierwszy jest producentem żądań metody, drugi ich konsumentem (przez Scheduler).
- **Scheduler** (Zarządca). Wykonuje się w osobnym wątku, zarządzając Activation Queue. Decyduje, które Method Request ma być zdjęte z kolejki i wykonane w Servant implementującym metodę. Ta decyzja oparta jest o różne kryteria, np. kolejność umieszczenia metod w kolejce, spełnienie pewnych warunków związanych z synchronizacją (np. zwolnienie miejsca w buforze). Uwarunkowania związane z synchronizacją są zwykle sprawdzane przy użyciu metod - strażników.

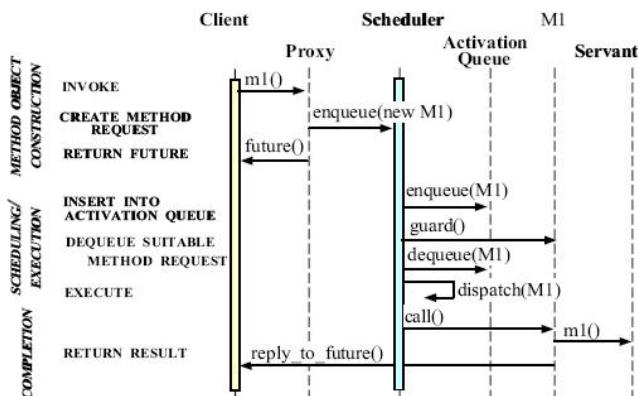
- **Future** (Zmienna terminowa). Pozwala klientowi pobranie wyniku wykonania metody, gdy Servant zakończy jej wykonanie. Po wywołaniu metody w Proxy, obiekt Future jest od razu zwracany. Future rezerwuje miejsce na wyniki wywołania metody. Gdy klient chce te wyniki pobrać, albo się blokuje do czasu pojawienia się wyników, albo periodycznie sprawdza przez polling.

Struktura Active Object wygląda następująco:



Rysunek 1: Struktura Active Object

Natomiast przebieg wykonania w ten sposób:



Rysunek 2: Schemat wykonania wzorca Active Object

3 Implementacja wzorca Active Object w problemie Producentów i konsumentów

Implementację wykonałem w języku Java. Na potrzeby programu dokonałem niezbędnych importów, które przedstawia kod (1). Część z nich stanowi jedynie pomoc w wykonaniu programu z różnymi parametrami i zapis wyników wydajności do pliku.

```

1 import java.util.concurrent.BlockingQueue;
2 import java.util.concurrent.LinkedBlockingQueue;

```

```

3 import java.util.LinkedList;
4 import java.util.Queue;
5 import java.io.FileWriter;
6 import java.io.IOException;
7 import java.util.ArrayList;
8 import java.util.List;

```

Kod Java 1: Zaimportowane elementy języka Java

3.1 Klasa PerformanceLogger

Klasa **PerformanceLogger** jest klasą pomocniczą realizującą zapisywanie konfiguracji uruchomienia oraz czasu wykonania w zależności od parametrów. Blok (2) przedstawia kod omawianej klasy.

```

1 class PerformanceLogger {
2     private final String fileName;
3     public PerformanceLogger(String fileName) {
4         this.fileName = fileName;
5     }
6     public void log(String configuration, int bufferSize, long duration) {
7         try (FileWriter writer = new FileWriter(fileName, true)) {
8             writer.write(configuration + "," + bufferSize + "," + duration + "\n");
9         } catch (IOException e) {
10             e.printStackTrace();
11         }
12     }
13     public void init() {
14         try (FileWriter writer = new FileWriter(fileName)) {
15             writer.write("Producers,Consumers,BufferSize,Duration (ms)\n");
16         } catch (IOException e) {
17             e.printStackTrace();
18         }
19     }
20 }

```

Kod Java 2: Zaimportowane elementy języka Java

3.2 Klasa Main

Klasa **Main** zajmuje się tworzeniem wszystkich obiektów wykorzystywanych we wzorcu Active Object, w zależności od testowanej konfiguracji. W celu poprawy czytelności tworzenie i nadzorowanie pracy wątków producentów i konsumentów rozdzieliłem na drugą funkcję *runTest*.

```

1 public class Main {
2     public static void main(String[] args) {
3         PerformanceLogger logger = new PerformanceLogger("performance_results.csv");
4         logger.init();
5         int[] producerCounts = {1, 2, 5, 10, 20, 50, 100};
6         int[] consumerCounts = {1, 2, 5, 10, 20, 50, 100};
7         for (int bufferSize : new int[]{1,2, 5, 10, 20, 50, 100}) {
8             Servant servant = new Servant(bufferSize);
9             ActivationQueue activationQueue = new ActivationQueue();
10            Proxy proxy = new Proxy(servant, activationQueue);
11            Scheduler scheduler = new Scheduler(activationQueue);
12            Thread schedulerThread = new Thread(scheduler);
13            schedulerThread.start();
14            for (int producers : producerCounts) {
15                for (int consumers : consumerCounts) {
16                    System.out.printf("Testing %d producers, %d consumers, buffer size %d...\n", producers, consumers, bufferSize);
17                    long duration = runTest(proxy, producers, consumers);
18                    logger.log(producers + "," + consumers, bufferSize, duration);
19                }
20            }
21        }
22    }
23 }

```

```

19         }
20     }
21     schedulerThread.interrupt();
22 }
23
24 private static long runTest(Proxy proxy, int producerCount, int consumerCount) {
25     List<Thread> threads = new ArrayList<>();
26     long startTime = System.currentTimeMillis();
27     for (int i = 0; i < producerCount; i++) {
28         Thread producer = new Thread(() -> {
29             for (int j = 0; j < 100; j++) {
30                 proxy.put(j);
31                 try {
32                     Thread.sleep(10);
33                 } catch (InterruptedException e) {
34                     Thread.currentThread().interrupt();
35                 }
36             }
37         });
38         threads.add(producer);
39         producer.start();
40     }
41     for (int i = 0; i < consumerCount; i++) {
42         Thread consumer = new Thread(() -> {
43             for (int j = 0; j < 100; j++) {
44                 proxy.take();
45                 try {
46                     Thread.sleep(15);
47                 } catch (InterruptedException e) {
48                     Thread.currentThread().interrupt();
49                 }
50             }
51         });
52         threads.add(consumer);
53         consumer.start();
54     }
55     for (Thread thread : threads) {
56         try {
57             thread.join();
58         } catch (InterruptedException e) {
59             Thread.currentThread().interrupt();
60         }
61     }
62     long endTime = System.currentTimeMillis();
63     return endTime - startTime;
64 }
65 }

```

Kod Java 3: Klasa Main

3.3 Klasa Proxy

Kod (4) przedstawia implementację klasy **Proxy**. Reprezentuje interfejs dostępu do asynchronicznych metod serwanta. Przyjmuje żądania (put/take) i umieszcza je w kolejce aktywacji, gdzie zostaną później przetworzone.

```

1 class Proxy {
2     private final Servant servant;
3     private final ActivationQueue activationQueue;
4
5     public Proxy(Servant servant, ActivationQueue activationQueue) {

```

```

6         this.servant = servant;
7         this.activationQueue = activationQueue;
8     }
9
10    public void put(int value) {
11        activationQueue.enqueue(new PutRequest(servant, value));
12    }
13
14    public void take() {
15        activationQueue.enqueue(new TakeRequest(servant));
16    }
17 }

```

Kod Java 4: Klasa **Proxy**

3.4 Klasa **Servant**

Kod (5) przedstawia implementację klasy **Servant**. Reprezentuje obiekt wykonujący rzeczywistą logikę biznesową. Przechowuje dane w buforze o ograniczonej pojemności i oferuje metody `put` (dodanie elementu) oraz `take` (pobranie elementu).

```

1 class Servant {
2     private final Queue<Integer> buffer;
3     private final int capacity;
4
5     public Servant(int capacity) {
6         this.buffer = new LinkedList<>();
7         this.capacity = capacity;
8     }
9
10    public void put(int value) {
11        buffer.add(value);
12    }
13
14    public int take() {
15        return buffer.poll();
16    }
17
18    public boolean isFull() {
19        return buffer.size() >= capacity;
20    }
21
22    public boolean isEmpty() {
23        return buffer.isEmpty();
24    }
25 }

```

Kod Java 5: Klasa **Servant**

3.5 Klasa **Method Request**

Kod (6) przedstawia implementację klasy **Method Request** jak klasy abstrakcyjnej oraz dziedziczących po niej klasach **PutRequest** oraz **TakeRequest**. Klasa po której dziedziczą definiuje metody `guard` (warunki synchronizacji) i `execute` (wykonanie logiki żądania). Klasy dziedziczące mogą umieszczać obiekty w byforze jeśli nie jest pełny oraz zabierać jeśli nie jest pusty.

```

1 abstract class MethodRequest {
2     protected final Servant servant;
3     public MethodRequest(Servant servant) {
4         this.servant = servant;
5     }

```

```

6     public abstract boolean guard();
7     public abstract void execute();
8 }
9 class PutRequest extends MethodRequest {
10     private final int value;
11     public PutRequest(Servant servant, int value) {
12         super(servant);
13         this.value = value;
14     }
15     @Override
16     public boolean guard() {
17         return !servant.isFull();
18     }
19     @Override
20     public void execute() {
21         servant.put(value);
22         System.out.println("Wstawiono: " + value);
23     }
24 }
25 class TakeRequest extends MethodRequest {
26     public TakeRequest(Servant servant) {
27         super(servant);
28     }
29     @Override
30     public boolean guard() {
31         return !servant.isEmpty();
32     }
33     @Override
34     public void execute() {
35         int value = servant.take();
36         System.out.println("Pobrano: " + value);
37     }
38 }

```

Kod Java 6: Klasy **MethodRequest**, **PutRequest**, **TakeRequest**

3.6 Klasa **ActivationQueue**

Kod (7) przedstawia implementację klasy **ActivationQueue**. Kolejka synchronizująca żądania (instancje **MethodRequest**). Zarządza kolejnością ich przetwarzania i zapewnia bezpieczeństwo wątków.

```

1 class ActivationQueue {
2     private final BlockingQueue<MethodRequest> queue = new LinkedBlockingQueue<>();
3     public void enqueue(MethodRequest request) {
4         try {
5             queue.put(request);
6         } catch (InterruptedException e) {
7             Thread.currentThread().interrupt();
8         }
9     }
10    public MethodRequest dequeue() {
11        try {
12            return queue.take();
13        } catch (InterruptedException e) {
14            Thread.currentThread().interrupt();
15            return null;
16        }
17    }
18 }

```

Kod Java 7: Klasa **ActivationQueue**

3.7 Klasa Scheduler

Kod (8) przedstawia implementację klasy **Scheduler**. Działa jako wątek wykonawczy, przetwarzający żądania z kolejki aktywacji. Sprawdza warunki synchronizacji (guard) dla każdego żądania i wykonuje je, jeśli są spełnione. Jeśli nie, żądanie wraca do kolejki.

```
1 class Scheduler implements Runnable {
2     private final ActivationQueue activationQueue;
3     public Scheduler(ActivationQueue activationQueue) {
4         this.activationQueue = activationQueue;
5     }
6     @Override
7     public void run() {
8         while (true) {
9             try {
10                MethodRequest request = activationQueue.dequeue();
11                if (request != null && request.guard()) {
12                    request.execute();
13                } else {
14                    activationQueue.enqueue(request);
15                }
16            } catch (Exception e) {
17                Thread.currentThread().interrupt();
18                break;
19            }
20        }
21    }
22 }
```

Kod Java 8: Klasa **Scheduler**

4 Wnioski

Zastosowanie wzorca Active Object w implementacji pozwoliło na skuteczne oddzielenie logiki przetwarzania żądań od mechanizmu ich synchronizacji. Dzięki temu system obsługuje wielu producentów i konsumentów w sposób asynchroniczny, co minimalizuje ryzyko kolizji przy współdzieleniu zasobów, takich jak bufor. Kluczowym elementem jest tu kolejka aktywacyjna, która umożliwia przechowywanie i przetwarzanie żądań w odpowiedniej kolejności, uwzględniając warunki synchronizacji zdefiniowane w metodach guard(). Scheduler pełni rolę zarządzającą, stale monitorując kolejkę i uruchamiając tylko te żądania, które mogą być natychmiast wykonane. Taki podział odpowiedzialności poprawia czytelność kodu, ułatwia rozszerzanie funkcjonalności i redukuje złożoność zarządzania wielowątkowością. Wzorec ten sprawdził się w testowanych konfiguracjach, zapewniając stabilność działania niezależnie od liczby producentów, konsumentów i rozmiaru bufora. System zachowuje się przewidywalnie, a mechanizmy asynchroniczne pozwalają na równoczesne przetwarzanie wielu żądań, minimalizując blokady i maksymalizując wykorzystanie dostępnych zasobów. W rezultacie zastosowanie wzorca Active Object okazało się skuteczne w budowie systemu wielowątkowego, zapewniając zarówno bezpieczeństwo współbieżne, jak i skalowalność.

Literatura

- [1] https://pl.wikipedia.org/wiki/Active_object
- [2] <https://java-design-patterns.com/patterns/active-object/>
- [3] <https://www.dre.vanderbilt.edu/~schmidt/PDF/Active-Objects.pdf>